

ARRAYS []

Class 13:

30/04/2025

Package com;

↳ ~~two~~ used []

↳ Group of values.

public class Test {

public static void main (String [] args) {

double st1 = 5.0;

double st2 = 7.5;

double st3 = 6.8;

double st3 = 9.9;

}

}

→ With same datatype, ~~we~~ of different variable we can use Arrays.

- Arrays store multiple values of same datatype with different variable.
- Arrays only accept same datatypes.
- Arrays come with a fixed size, doesn't work with different sizes.
* shows how many values it can hold.

→ Syntax:

① int [] arr = new int [size];

⇒ datatype [] arrayname = new datatype [size];

→ Array starts with 0th index.

Ex: arrs

0	1	2	3	4
1	10	5	3	2

(2) ⇒ int arr [] = new int [size];

arr [0] = 1

arr [3] = 3

arr [1] = 10

arr [4] = 2

arr [2] = 5

Ex: int [] arr = new int [10]

→ size : 10

→ Index : 0-9

5	0	0	0	0	0	0	0	0	6
---	---	---	---	---	---	---	---	---	---

→ arr [0] = 5

→ arr [9] = 6

↳ default values.

If we give, arr [0] = 7, it can update.

② `int [ ] arr = { 1, 10, 5 };`

→ size : 3

→ `print (arr[1]);` → output : 10

Ex: package com;

public class Test {

public static void main (String [] args) {

int [] arr = new int [5];

System.out.println (arr [0]); output : 0

arr [0] = 10

`syso ( arr [ 0 ] );` → output : 10 [arrays updated].

* Arrays take default values based on datatype, if we not given any values.

② `int [ ] arr2 = { 10, 15, 25, 35 };`

`syso ( arr2.length );` → tells size of array.

→ output : 4

`syso ( arr.length );` → output : 5

Ex:

~~void~~ void createCharArray () {

char [] arr = new char [size];

`syso ( "char array : " + arr [ 0 ] );`

arr [0] = 'A';

}

public static void main (String [] args) {

Test obj = new Test ();

obj.createCharArray ();

}

output : Char array : _

output : Char array : A


```

* void createStringArray ( ) {
    String [ ] array = new String [ 3 ];
    Syso ( " String array : " + array [ 0 ] );
    array [ 0 ] = " Hi ";
    Syso ( " String array : " + array [ 0 ] );
}

public static void main (String [ ] args) {
    Test obj = new Test ( );
    obj.createStringArray ( );
}

```

output: String array : null (default Value)
 String array : Hi (Updated Value)

Ex:

```

package com;
public class Arrays {
    public static void main (String [ ] args) {
        double [ ] marks = new double [ 5 ];
        marks [ 0 ] = 10.0;
        marks [ 1 ] = 6.7;
        marks [ 2 ] = 9.7;
        marks [ 3 ] = 6.1;
        marks [ 4 ] = 8.2;
        Syso (marks [ 0 ] );
        Syso (marks [ 1 ] );
        Syso (marks [ 2 ] );
        Syso (marks [ 3 ] );
        Syso (marks [ 4 ] );
    }
}

```

output: 10.0
 6.7
 9.7
 6.1
 8.2

default value: 0.0

```

double [ ] marks = new double [ 100 ];
for (int i = 0 ; i < marks.length ; i++) {
    Syso (marks [ i ] );
}

```

⇒ output: 10.0
 6.7
 9.7
 6.1
 8.2
 0.0
 0.0 ——— 0.0

^{Array}
`double [ ] marks = new double [ 100 size ];`
`Scanner sc = new Scanner (System.in);`
`syso ("Enter no. of Students : ");`
`int size = sc.nextInt ();`
`{`
`marks [0] = sc.nextDouble ();`
`marks [1] = sc.nextDouble ();`
`marks [2] = sc.nextDouble ();`
`⋮`
`for (int i = 0; i < marks.length; i++) {`
`syso ("Enter marks of Student " + (i+1) );`
`marks[i] = sc.nextDouble ();`
`}`

`for (int i = 0; i < marks.length; i++) {`
`syso ("Marks of student " + (i+1) + " → " + marks[i]);`
`}`

output:

Enter marks of student 1

7.1

Marks of student 1 → 7.1

Output: Enter no. of students : 3

Enter marks of student 1

1.0

Enter marks of student 2

5.7

Enter marks of student 3

7.5

• Marks of student 1 → 1.0

" 2 → 5.7

" 3 → 7.5

* We can create an array with negative size (we can compile), but we got exception during running the code.

`int [ ] arr4 = new int [-1];` → we can't run, if we run, we will get exception.

⇒ Negative Array ^{size} ~~size~~ exception

* Arrays: To store multiple data.

Largest & Smallest Number

Class: 14

05/05/2025

package com;

import java.util.Scanner;

public class LargestAndSmallest { [used to store multiple values]

public static void main (String[] args) {

int[] arr = {10, 15, 20, 24, 15, 7, 13}; [JVM calling this method internally. (default)]
~~LargestAndSmallest obj = ob~~
LargestAndSmallest.findLargestAndSmallest(arr);

}

static void findLargestAndSmallest (int[] arr) {

int max = arr[0];

int min = arr[0];

for (int i = 0; i < arr.length; i++) {

if (arr[i] > max) {

max = arr[i];

}

~~else~~ if (arr[i] < min) {

min = arr[i];

}

}

System.out.println(max);

System.out.println(min);

Output: Largest: 24

Smallest: 7

* Reverse all Values in an array:

```
package com;  
import java.util.Scanner;  
public class ReverseArray {  
    PSVM {  
        int arr[] = {24, 35, 56, 78, 90};  
        int[] newArr = ReverseArray.reverse(arr);  
        sysout (newArr) for (int a : newArr) {  
            sysout(a);  
        }  
    }  
    static void reverse (int[] arr) {  
        int j = 0; int[] newArr = new int[arr.length];  
        for (int i = arr.length - 1; i >= 0; i--) {  
            newArr[j] = arr[i];  
            j++;  
        }  
        sysout return newArr;  
    }  
}
```

for each loop → also called enhanced for loop.

Syntax: for (int a : newArr) {

}

[* we can't print arrays directly, we have to ^{use} ~~each~~ foreach loop (&) enhanced for loop]

Output:

90

78

56

35

24


```

Ex: package com;
    public class ReverseArray {
        psum {
            int arr = { 24, 35, 56, 78, 90 }
        } int new Arr ReverseArray.reverseUsingWhile(arr);
        Static void reverseUsingWhile (int[] arr) {
            int start = 0;
            int end = arr.length - 1;
            while (start < end) {
                int temp = arr[start];
                arr[start] = arr[end];
                arr[end] = temp;
                start++;
                end--;
            }
            for (int a : arr) { // for Each loop / Enhance for loop.
                syso(a);
            }
        }
    }

```

output: 90
78
56
35
24

[* We can swap the values in an array.]

* Contagious Memory Location:

- Java will work in continuous memory location.
- It allows memory according to size of an array.
- Use: Read operations are faster in arrays due to this memory address.
- Base memory location + [(datatype used memory) × Index] = Memory address
- When we accessing values through index, internally CPU uses above formula and return the values.
- This makes the arrays very much faster due to it's values stores in contagious memory and fetches the values quickly with that memory address.

very much faster in Linked List.

* Write operations in array: Very much slower

→ If we want to shuffle the values in array, write operations are very much slow, so that it will take lot of time, so that this operations are not preferred in java.

→ Only read operations are preferred in arrays, due to it is very faster.

* Split Method: [String Method] - to divide the parts of string.

Ex: package com;

```
public class SplitMethod {
```

```
    public static void main (String[] args) {
```

```
        String s = "Hi This is Java Class";
```

```
        String[] words = s.split(" ");
```

↳ [to store multiple words]

[// String can hold only one word, to hold multiple words we have to use String[] array]

→ // we To print that words, we have to

use for-each loop.

```
        for (String word : words) {
```

```
            System.out.println(word);
```

```
        }
```

```
        System.out.println(Arrays.toString(words));
```

```
    }
```

```
}
```

Output: [Hi This is Java Class]

Output:

Hi
This
is

Java
Class.

[* we can split individual letters of a word also]

* To split ~~word~~ words & individual letters in a word, we have common thing i.e like space.

* Arrays class: gives default methods.

1. Arrays.toString(words) $\Rightarrow$ we can directly print the array without using for each loop.
(Static method)

2D Arrays (Array of Arrays)

- $\rightarrow$ It is a type of row & column concept.
- $\rightarrow$ It works with a matrix type questions.
- $\rightarrow$ It can hold rows and column values at a time.

Syntax: 1) `int [ ][ ] arr = new int [Rowsize] [Columnsize];`
 2) `int [ ][ ] arr = {{1, 45, 67}, {45, 23, 10}, {20, 16, 35}};`

$R_0 C_0$	$R_0 C_1$
$R_1 C_0$	$R_1 C_1$

$arr[0][0] = 5$
 $arr[0][1] = 10$
 $arr[1][0] = 6$
 $arr[1][1] = 2$

$\Rightarrow$ to insert & read the values in 2D array.

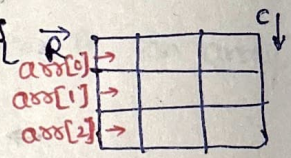
$\rightarrow$ Have same no. of rows & columns & every individual row is also an array.

Ex: package com;

public class TwoDArray {

public static void main(String[] args) {

int [][] arr = new int [3] [3];



we can use, Nested for loop.

$arr[0][0] = 10$	$arr[2][0] = 25$
$arr[0][1] = 20$	$arr[2][1] = 20$
$arr[0][2] = 50$	$arr[2][2] = 30$
$arr[0][3] = 30$	$arr[2][3] = 15$
$arr[1][0] = 5$	$arr[3][0] = 18$
$arr[1][1] = 10$	$arr[3][1] = 22$
$arr[1][2] = 63$	$arr[3][2] = 26$
$arr[1][3] = 60$	$arr[3][3] = 32$

(Shows no. of rows)

for (int i=0; i < arr.length, i++) {
 for (int j=0; j < arr[i].length, j++)
 syso (arr[i][j]);
 }

}
Output:
 10
 20
 50 5 10 63 25 20 30

* Jagged Array: Array has different no. of rows & columns.

~~int arr[3][2]~~ → arr[3][2]

(2D Array)

for (int [] a : arr) {

System.out.println(Arrays.toString(a));

}

output: [10 20 50]

[5 10 63]

[25 20 30]

class: 16

07/05/2025